

A decorative graphic on the left side of the cover, consisting of a dark grey vertical band. Overlaid on this band is a light blue circuit board pattern. The pattern features several vertical lines representing traces, with various horizontal and diagonal lines branching off. Small circles, representing vias or components, are placed at the ends of these lines and at various points along the traces.

## **Autori**

Franchetti Carlo Maria

Di Lieto Mario

Di Lieto Gabriele

# SMART PET FEEDER

# PROBLEMA

01

Qual è il problema  
che si vuole  
risolvere?

02

Perché è utile un  
dispenser  
intelligente?

03

Target di utenti  
(proprietari di  
animali, persone  
impegnate, ecc.)



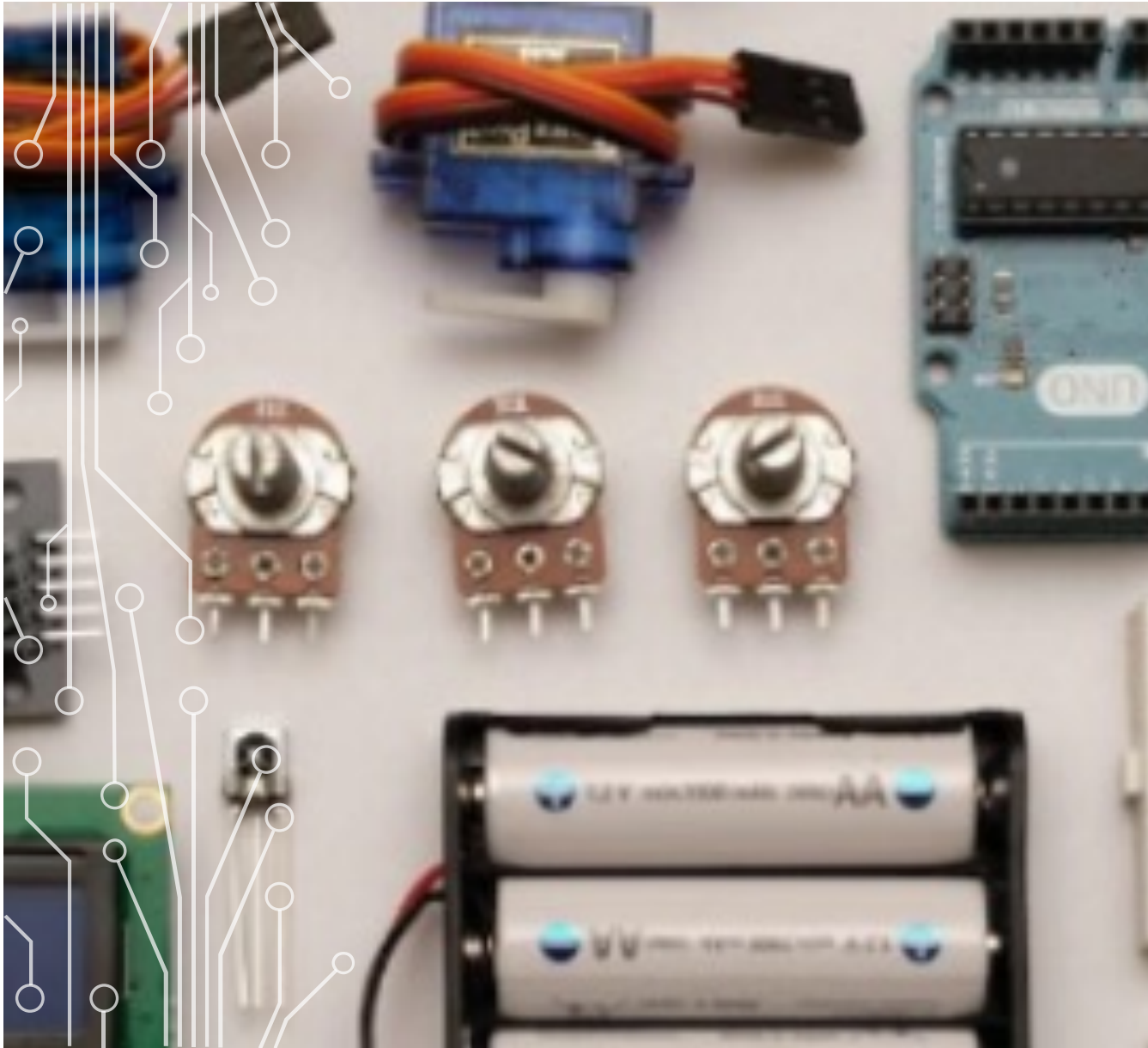
## SOLUZIONE

Funzionalità principali:

- Erogazione automatica programmata
- Erogazione manuale in caso di problemi
- Sensori di prossimità e di accumulo
- Allarmi e notifiche

# ARCHITETTURA DEL SISTEMA

- **ESP32** connessa via Wi-Fi, gestisce logica e attuatori.
- **Broker MQTT (HiveMQ)** per comunicazione con la dashboard.
- **Database PostgreSQL (pet\_feeder)** per la persistenza dei dati.
- **Dashboard web** per gestione, visualizzazione e analisi.



## COMPONENTI HARDWARE

- ESP32
- Motore per erogazione
- Sensore IR (rilevamento accumulo)
- Sensore di prossimità
- Buzzer
- Connessione Wi-Fi

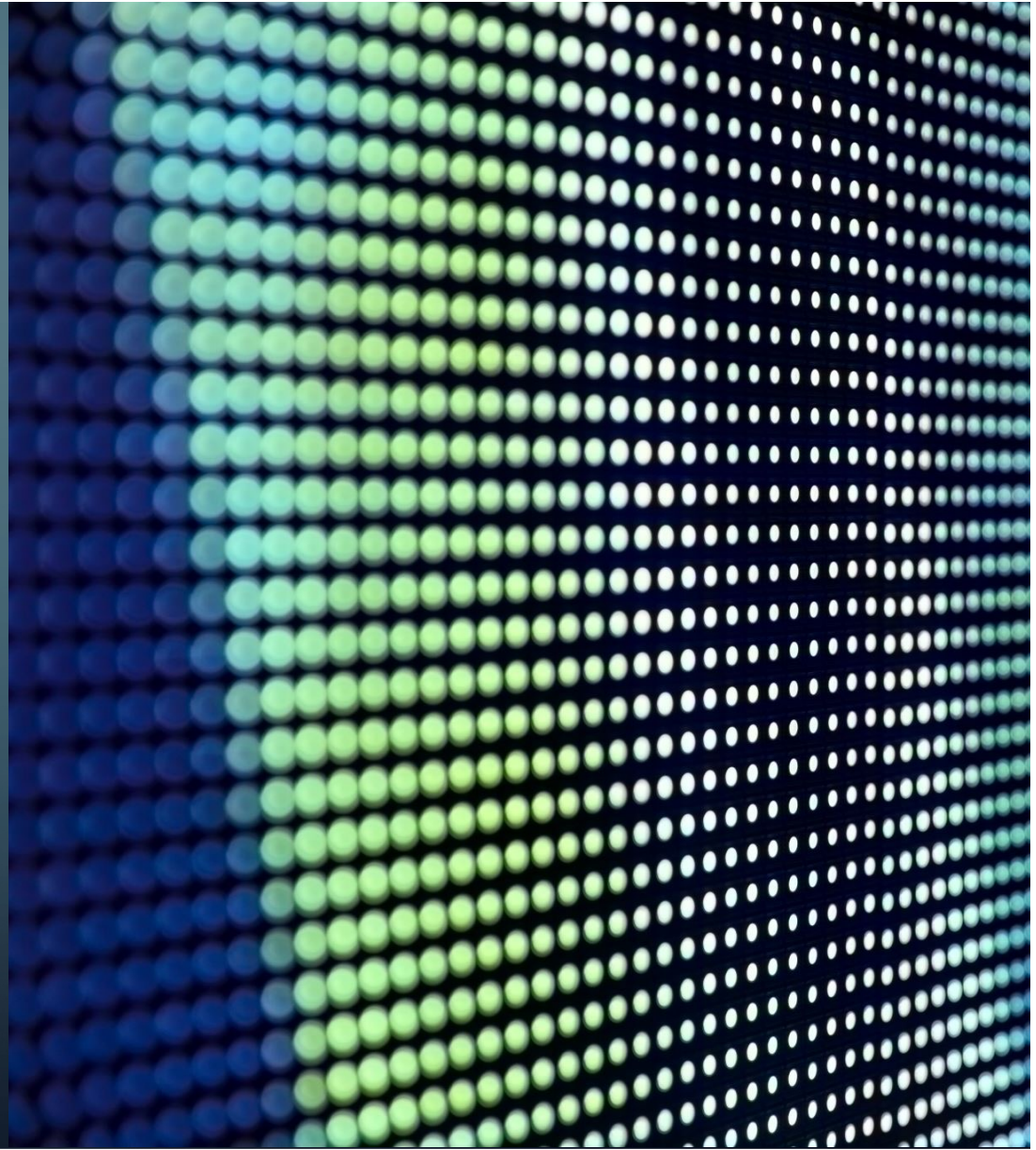


A decorative graphic on the left side of the slide, featuring a dark teal background with white lines and circles representing a circuit board or network topology.

# SOFTWARE & TECNOLOGIE

Linguaggi e strumenti usati:

- MycroPython (ESP32)
- PHP, HTML, JS, CSS (interfaccia web)
- PostgreSQL (database pet\_feeder)
- HiveMQ (broker MQTT)



# CODICE PYTHON: SENSORS

## IR\_Sensor

```
class IRSensor:
    def __init__(self, pin_num):
        self.sensor = Pin(pin_num, Pin.IN)

    def is_obstructed(self):
        return self.sensor.value() == 0
```

## Ultrasonic\_Sensor

```
class UltrasonicSensor:
    def __init__(self, trig_pin, echo_pin):
        self.trig = Pin(trig_pin, Pin.OUT)
        self.echo = Pin(echo_pin, Pin.IN)

    def read_distance(self):
        self.trig.value(0)
        time.sleep_us(2)
        self.trig.value(1)
        time.sleep_us(10)
        self.trig.value(0)
        while self.echo.value() == 0:
            start = time.ticks_us()
        while self.echo.value() == 1:
            end = time.ticks_us()
        duration = time.ticks_diff(end, start)
        return (duration / 2) / 29.1 # cm
```

# CODICE PYTHON: CONTROLLER

```
def check_food_level(self):
    try:
        dist = self.ultra_food.read_distance()
        print("Livello croccantini:", dist)
        if dist > 15:
            print("Livello croccantini basso!")
            self.mqtt.publish_alert(b'Livello croccantini basso!')
    except:
        pass
```

```
def execute_dispense(self):
    self.buzzer.start_sound()
    self.servo.dispense()
    if self.ir.is_obstructed():
        self.buzzer.start_sound()
        print("Limite croccantini raggiunto!")
        show_ir_alert()
        self.mqtt.publish_alert(b'Accumulo rilevato!')
    self.mqtt.publish_status(b'Erogazione completata.')
    print("Erogazione completata.")
    show_dispense_message()
```

```
def check_proximity(self):
    global scheduled_time

    try:
        dist = self.ultra_animal.read_distance()
        print("Distanza animale:", dist)
        if dist < 10:
            print("Animale rilevato")

            now = time.localtime(time.time() + 3600 * 2)
            hour, minute, second = now[3], now[4], now[5]
            now_secs = hour * 3600 + minute * 60 + second
            now_str = f"{hour:02d}:{minute:02d}"

            for sched in scheduled_time:
                try:
                    sched_time = sched['time'][:5]
                    if sched_time in self.erogazioni_anticipate:
                        continue

                    h, m = map(int, sched_time.split(":"))
                    sched_secs = h * 3600 + m * 60
                    diff = sched_secs - now_secs

                    if 0 < diff <= 600:
                        print(f"Animale rilevato entro 10 minuti da erogazione programmat")
                        self.mqtt.publish_status(f"Erogazione anticipata per rilevamento")
                        show_early_dispense()
                        self.execute_dispense()

                        self.erogazioni_anticipate.add(sched_time)
                        time.sleep(10)
                        break # evita doppia erogazione nel singolo ciclo
                    except Exception as e:
                        print("Errore parsing orario programmato:", e)
            except Exception as e:
                print("Errore rilevamento prossimità:", e)
```



# CODICE PYTHON: MQTT\_MANAGER

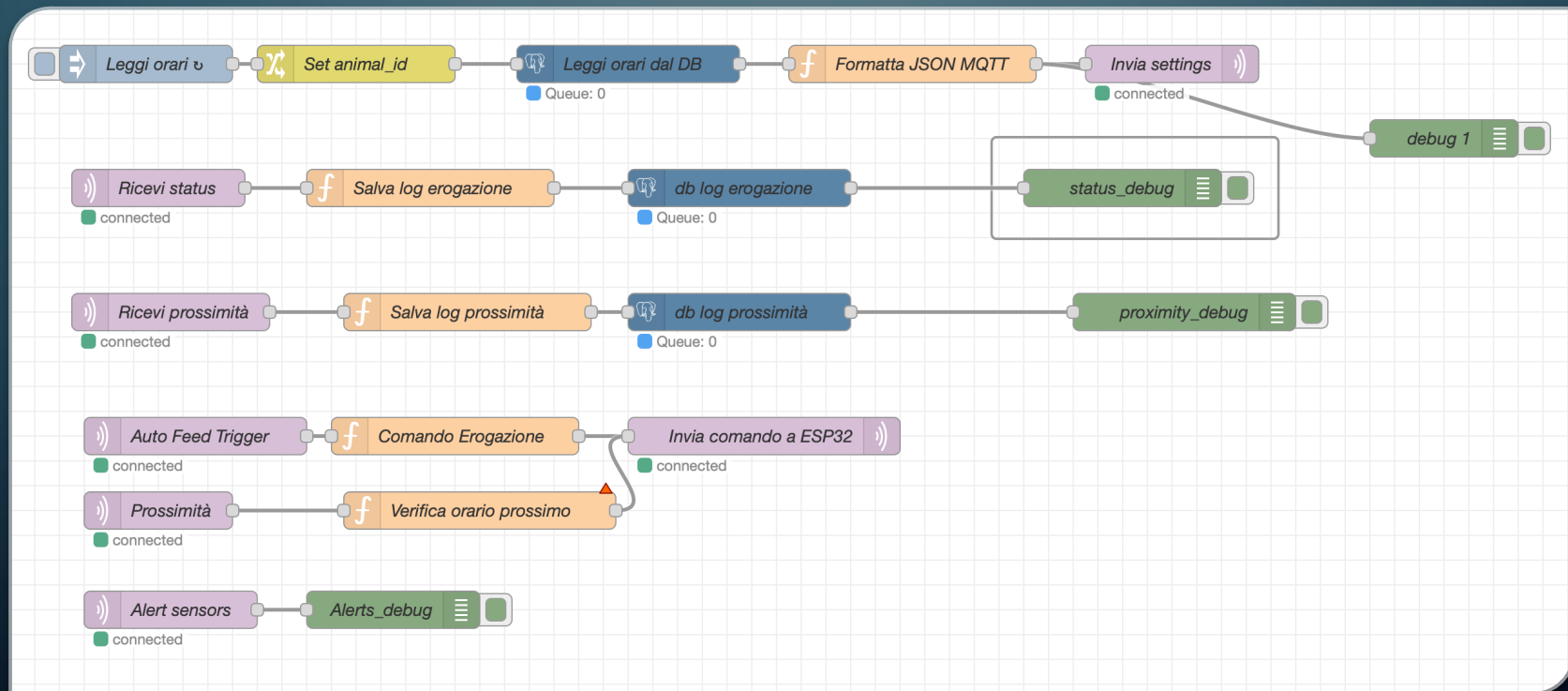
```
def connect(self):  
    self.client.connect()  
    self.client.subscribe(self.topic_cmd)  
    self.client.subscribe(b'petfeeder/schedule')  
  
def check_msg(self):  
    self.client.check_msg()  
  
def publish_status(self, msg):  
    self.client.publish(self.topic_status, msg)  
  
def publish_alert(self, msg):  
    self.client.publish(self.topic_alert, msg)
```

La classe MQTTManager è progettata per gestire la comunicazione tra il dispositivo e il broker MQTT.

Quando viene istanziata, inizializza un client MQTT con un identificativo specifico, l'indirizzo del broker e i nomi dei topic su cui il dispositivo dovrà pubblicare o ricevere messaggi.

Inoltre, collega al client una funzione di callback, che verrà eseguita ogni volta che arriverà un nuovo messaggio su uno dei topic sottoscritti.

# TEST E RISULTATI



# FUNZIONALITÀ DELLA DASHBOARD WEB



Login/Registrazione  
utente



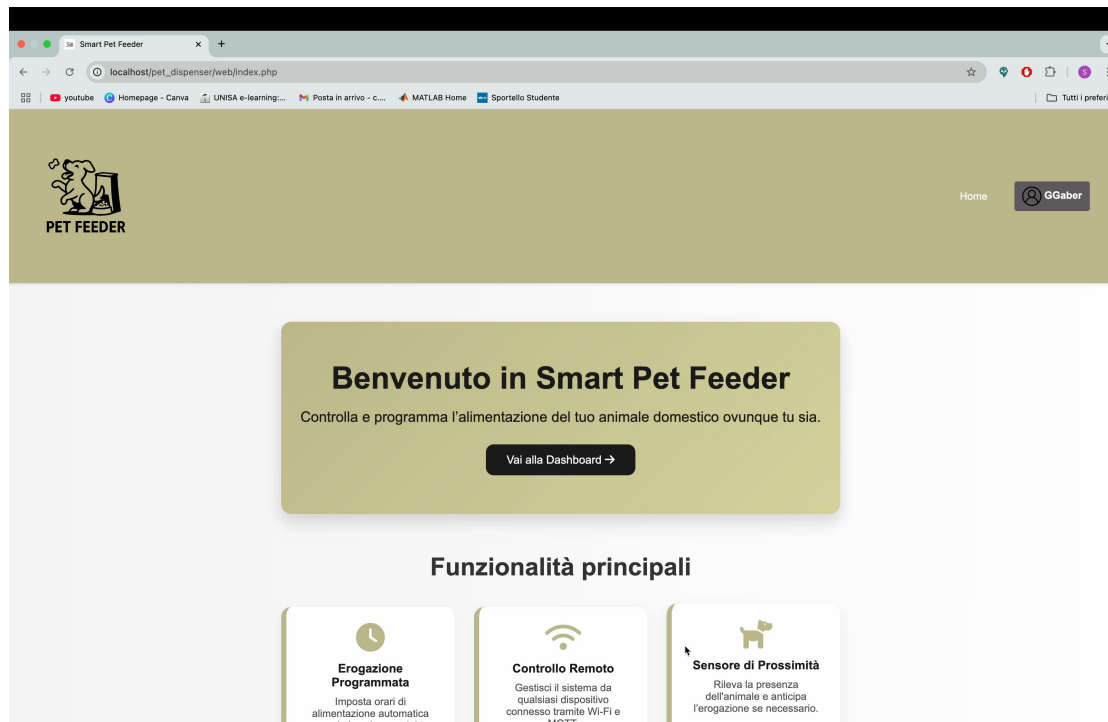
Gestione animali



Programmazione orari



Storico erogazioni



## UTILIZZO DEL SITO WEB

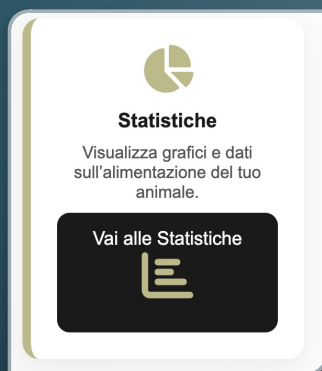
Il sito web del progetto funge da interfaccia per monitorare e gestire l'erogazione automatica del cibo per animali domestici.

Consente agli utenti di visualizzare dati come gli orari di erogazione, la quantità di cibo erogata e i rilevamenti dei sensori.

Fornisce suggerimenti per ottimizzare l'uso del dispenser, in base alla frequenza e agli orari di avvicinamento dell'animale al dispositivo.



# ANALISI STATISTICHE



## Consigli per migliorare l'erogazione del cibo

L'animale si avvicina poco al dispenser: prova a spostarlo in un posto più accessibile o visibile.

Metriche analizzate (ultimi 7 giorni):  
Totale rilevamenti di prossimità: 0  
Rilevamenti fuori orari programmati: 0

Monitoraggio del comportamento dell'animale rispetto al dispenser.

Si basano su:

- Totale rilevamenti di prossimità - Storico
- Rilevamenti fuori orari programmati
- Suggerimenti personalizzati - Suggerimenti

# CONCLUSIONI & SVILUPPI FUTURI

## Potenziali miglioramenti

- Gestione multi-animale
- Aggiunta lettore NFC per identificazione animale specifico
- Personalizzazione per tipo di animale
- Controllo della quantità dei croccantini erogati, tramite sensore di peso
- Rilevazione animale per un periodo di tempo prolungato